LAB 7
Fuzz Testing for Input Validation Bugs

Visa Application - Hypothesis Property-Based Testing

Student Name	Bharath Hegde
SRN	PES1UG24CS114
Section	B
Institution / Department	PES University - Computer Science and Engineering
Submission Date	11 November 2026

Repository: DuoCoder986/SE-Labs-PES1UG24CS114
Folder: Lab 7
View Lab 7 folder on GitHub

1. Objective and Lab Overview

The objective of this lab is to use fuzz testing to discover input-validation bugs, analyse the generated failing inputs, improve defensive input handling, and rerun tests to verify the corrections.

The assigned application is a visa application program. This report uses the faculty-required filenames `processor.py` and `test_processor.py`. The three functions under test are `parse_application()`, `processing_average()`, and `payable_fee()`.

2. Environment and Tools

Item	Recorded value
Operating system	Windows / win32
Python	3.12.10
pytest	9.1.1
Hypothesis	6.170.15
Commands	`python processor.py` `python -m pytest test_processor.py -v`

3. Functions Under Test

Function	Purpose	Valid sample	Output
`parse_application(text)`	Parses visa type and entry count separated by a colon.	`tourist:2`	`('Tourist', 2)`
`processing_average(days_csv)`	Calculates the integer average of comma-separated days.	`5,10,21`	`12`
`payable_fee(text)`	Adds 18% to the fee, subtracts discount, and floors at zero.	`6000|500`	`6580.0`

4. Fuzz Testing Methodology

The tests use Hypothesis' `@given` decorator with `st.text() | st.none()`. Hypothesis generates arbitrary strings and None values for each function. Each generated input is passed directly to the function so unexpected exceptions are reported as test failures.

5. Bugs Exposed by the Initial Fuzz Run

Function	Failing example	Observed issue
parse_application	`':'`	Empty entry count caused integer conversion ValueError.
parse_application	`''`	Missing colon caused tuple-unpacking ValueError.
parse_application	`None`	Calling split() on None caused AttributeError.
processing_average	`''`	Empty input could not be converted to integer.
processing_average	`None`	Calling split() on None caused AttributeError.
payable_fee	`'|'`	Empty fee/discount caused float conversion ValueError.
payable_fee	`''`	Missing pipe caused tuple-unpacking ValueError.
payable_fee	`None`	Calling split() on None caused AttributeError.

The initial pytest run collected three fuzz tests and all three failed. Hypothesis reported counterexamples involving missing delimiters, empty fields, and None values.

6. Defensive Fixes Applied

parse_application() checks the input type, requires one colon, rejects empty fields, validates integer conversion, and rejects non-positive entry counts.

processing_average() checks the input type and non-empty CSV, validates each integer element, and rejects negative day values.

payable_fee() requires one pipe separator, validates numeric conversion, rejects negative and non-finite values, and checks that the computed total is finite.

Invalid inputs return `None`. Valid inputs preserve the original output format and calculation behaviour.

7. Final Verification and Conclusion

After the defensive checks were added, the same three Hypothesis tests passed. Fifteen parameterized correctness cases were also added to check representative valid and invalid values.

Test group	Count	Result
Hypothesis no-crash tests	3	**3 passed**
Parameterized correctness cases	15	**15 passed**
Total final test run	18	**18 passed**

The repository evidence shows 18 collected tests and 18 passed in 2.06 seconds.

Conclusion

Hypothesis-based fuzz testing exposed input-validation failures in all three functions. Defensive validation was added, and the same fuzz tests passed after the corrections. The extended suite completed with 18 of 18 tests passing.

Submission requirements

This consolidated PDF includes the required execution screenshots and complete source listings for `processor.py` and `test_processor.py`. The Lab 7 folder is present in the GitHub repository. Upload this single PDF using the faculty form: Google Form submission link.

Evidence 1

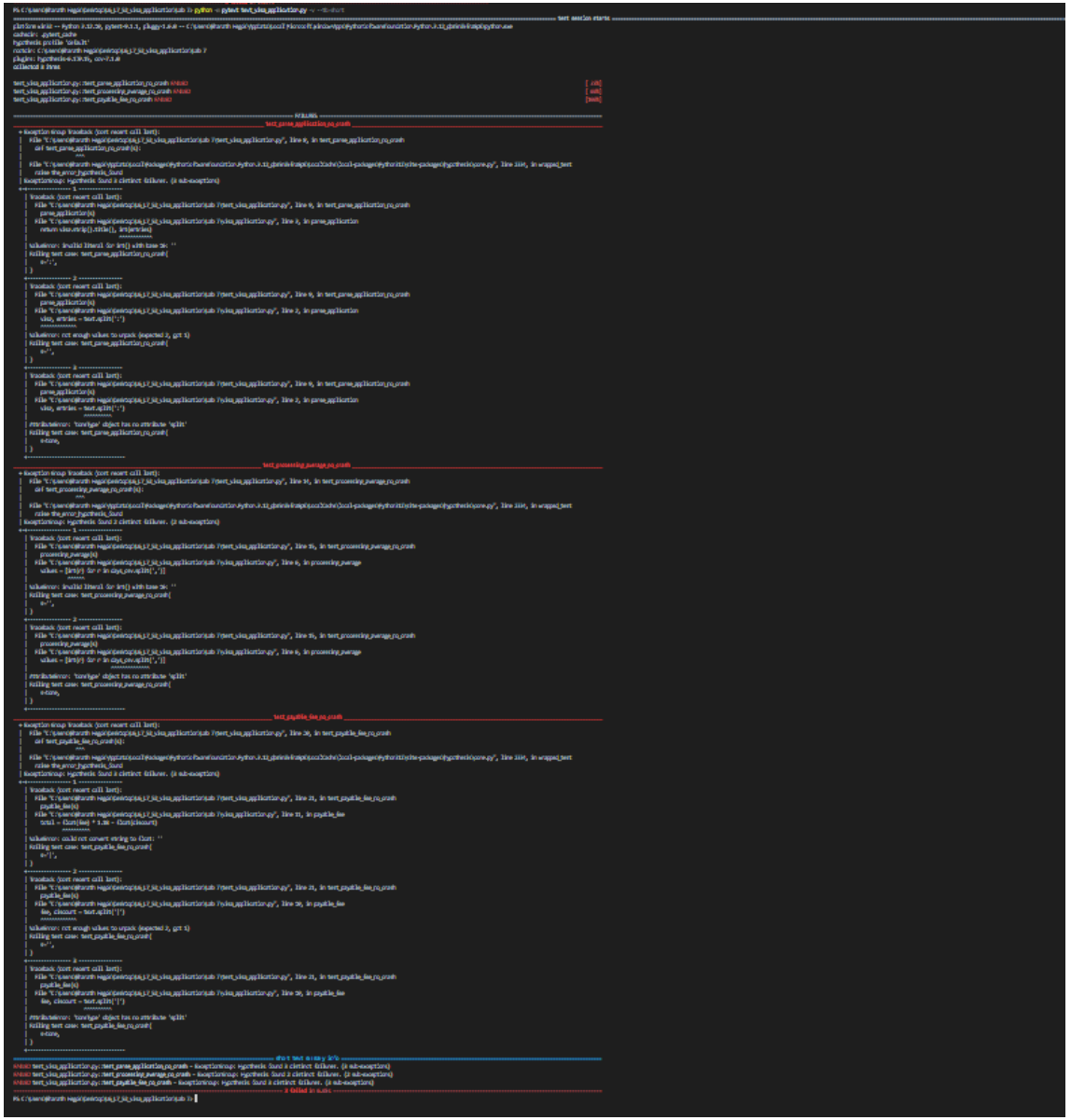
```
PS C:\Users\Bharath Hegde\Desktop\L6_L7_52_visa_application\Lab 7> python visa_application.py
==== Running parse_application ====
Enter visa type:entries (e.g. tourist:2): tourist:2
Visa Type and Entries: ('Tourist', 2)

==== Running processing_average ====
Enter processing days as CSV (e.g. 5,10,21): 5,10,21
Average Processing Days: 12

==== Running payable_fee ====
Enter fee|discount (e.g. 6000|500): 6000|500
Payable Fee: 6580.0
PS C:\Users\Bharath Hegde\Desktop\L6_L7_52_visa_application\Lab 7>
```

Evidence 1 - Original application executed successfully with valid sample inputs.

Evidence 2



Evidence 2 - Initial Hypothesis fuzz run. All three no-crash tests failed and reported counterexamples.

Evidence 3

```
PS C:\Users\Bharath Hegde\Desktop\L6_L7_52_visa_application\Lab 7> python visa_application.py
==== Running parse_application ====
Enter visa type:entries (e.g. tourist:2): tourist:2
Visa Type and Entries: ('Tourist', 2)

==== Running processing_average ====
Enter processing days as CSV (e.g. 5,10,21): 5,10,21
Average Processing Days: 12

==== Running payable_fee ====
Enter fee|discount (e.g. 6000|500): 6000|500
Payable Fee: 6580.0
PS C:\Users\Bharath Hegde\Desktop\L6_L7_52_visa_application\Lab 7>
```

Evidence 3 - Corrected application executed with the same valid sample inputs.

Evidence 4

```
PS C:\Users\Bharath Hegde\Desktop\L6_L7_52_visa_application\Lab 7> python -m pytest test_visa_application.py -v
============================================= test session starts =============================================
platform win32 -- Python 3.12.10, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\Bharath Hegde\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.12_qbz5n2kfra8p0\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\Bharath Hegde\Desktop\L6_L7_52_visa_application\Lab 7
plugins: hypothesis-6.170.15, cov-7.1.0
collected 3 items

test_visa_application.py::test_parse_application_no_crash PASSED                                    [ 33%]
test_visa_application.py::test_processing_average_no_crash PASSED                                   [ 66%]
test_visa_application.py::test_payable_fee_no_crash PASSED                                          [100%]

============================================== 3 passed in 1.65s ==============================================
PS C:\Users\Bharath Hegde\Desktop\L6_L7_52_visa_application\Lab 7>
```

Evidence 4 - The three Hypothesis no-crash tests passed after the fixes.

Additional Evidence

```
PS C:\Users\Bharath Hegde\Desktop\L6_L7_52_visa_application\Lab 7> python -m pytest test_visa_application.py -v
============================================= test session starts =============================================
platform win32 -- Python 3.12.10, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\Bharath Hegde\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.12_qbz5n2kfra8p0\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\Bharath Hegde\Desktop\L6_L7_52_visa_application\Lab 7
plugins: hypothesis-6.170.15, cov-7.1.0
collected 18 items

test_visa_application.py::test_parse_application_no_crash PASSED                                    [  5%]
test_visa_application.py::test_processing_average_no_crash PASSED                                   [ 11%]
test_visa_application.py::test_payable_fee_no_crash PASSED                                          [ 16%]
test_visa_application.py::test_parse_application_correctness[tourist:2-expected0] PASSED            [ 22%]
test_visa_application.py::test_parse_application_correctness[business:5-expected1] PASSED           [ 27%]
test_visa_application.py::test_parse_application_correctness[tourist:-None] PASSED                  [ 33%]
test_visa_application.py::test_parse_application_correctness[:2-None] PASSED                        [ 38%]
test_visa_application.py::test_parse_application_correctness[tourist:-1-None] PASSED                [ 44%]
test_visa_application.py::test_processing_average_correctness[5,10,21-12] PASSED                    [ 50%]
test_visa_application.py::test_processing_average_correctness[10,20-15] PASSED                      [ 55%]
test_visa_application.py::test_processing_average_correctness[5,,10-None] PASSED                    [ 61%]
test_visa_application.py::test_processing_average_correctness[-None] PASSED                         [ 66%]
test_visa_application.py::test_processing_average_correctness[-5,10-None] PASSED                    [ 72%]
test_visa_application.py::test_payable_fee_correctness[6000|500-6580.0] PASSED                      [ 77%]
test_visa_application.py::test_payable_fee_correctness[1000|200-980.0] PASSED                       [ 83%]
test_visa_application.py::test_payable_fee_correctness[abc|500-None] PASSED                         [ 88%]
test_visa_application.py::test_payable_fee_correctness[|-None] PASSED                               [ 94%]
test_visa_application.py::test_payable_fee_correctness[-100|0-None] PASSED                          [100%]

============================================== 18 passed in 2.06s ==============================================
PS C:\Users\Bharath Hegde\Desktop\L6_L7_52_visa_application\Lab 7>
```

Additional evidence - The extended test suite completed with 18 tests passing.

Appendix A - Corrected Application Source Code

Repository file: Lab 7/processor.py. Source reproduced directly from the uploaded repository ZIP.

```
import math


def parse_application(text):
    # Check input type
    if not isinstance(text, str):
        return None

    # Require exactly one colon
    if text.count(':') != 1:
        return None

    visa, entries = text.split(':')
    visa = visa.strip()
    entries = entries.strip()

    # Both fields must be present
    if not visa or not entries:
        return None

    # Validate entry count
    try:
        entries = int(entries)
    except ValueError:
        return None

    if entries <= 0:
        return None

    return visa.title(), entries


def processing_average(days_csv):
    # Check input type
    if not isinstance(days_csv, str):
        return None

    if not days_csv.strip():
        return None

    try:
        values = [int(r.strip()) for r in days_csv.split(',')]
    except ValueError:
        return None

    # Reject empty or negative values
    if not values or any(day < 0 for day in values):
        return None

    return sum(values) // len(values)
```

Appendix A - Corrected Application Source Code (continued)

Continuation of the same repository source file.

```
def payable_fee(text):
    # Check input type
    if not isinstance(text, str):
        return None

    # Require exactly one separator
    if text.count('|') != 1:
        return None

    fee, discount = text.split('|')

    try:
        fee = float(fee.strip())
        discount = float(discount.strip())
    except ValueError:
        return None

    # Reject negative and non-finite values
    if not math.isfinite(fee) or not math.isfinite(discount):
        return None

    if fee < 0 or discount < 0:
        return None

    total = fee * 1.18 - discount

    if not math.isfinite(total):
        return None

    return max(total, 0)


def main():
    print("==== Running parse_application ====")
    user_input = input("Enter visa type:entries (e.g. tourist:2): ")
    result = parse_application(user_input)
    print("Visa Type and Entries:", result if result is not None else "Invalid input")

    print("\n==== Running processing_average ====")
    user_input = input("Enter processing days as CSV (e.g. 5,10,21): ")
    result = processing_average(user_input)
    print("Average Processing Days:", result if result is not None else "Invalid input")

    print("\n==== Running payable_fee ====")
    user_input = input("Enter fee|discount (e.g. 6000|500): ")
    result = payable_fee(user_input)
    print("Payable Fee:", result if result is not None else "Invalid input")


if __name__ == "__main__":
    main()
```

Appendix B - Hypothesis and Correctness Test Source Code

Repository file: Lab 7/test_processor.py. Includes three Hypothesis fuzz tests and fifteen parameterized correctness cases.

```
from hypothesis import given, strategies as st
from processor import parse_application, processing_average, payable_fee


# Test 1: Fuzz parse_application
@given(st.text() | st.none())
def test_parse_application_no_crash(s):
    parse_application(s)


# Test 2: Fuzz processing_average
@given(st.text() | st.none())
def test_processing_average_no_crash(s):
    processing_average(s)


# Test 3: Fuzz payable_fee
@given(st.text() | st.none())
def test_payable_fee_no_crash(s):
    payable_fee(s)


import pytest


@pytest.mark.parametrize("input_data, expected", [
    ("tourist:2", ("Tourist", 2)),
    ("business:5", ("Business", 5)),
    ("tourist:", None),
    (":2", None),
    ("tourist:-1", None),
])
def test_parse_application_correctness(input_data, expected):
    assert parse_application(input_data) == expected


@pytest.mark.parametrize("input_data, expected", [
    ("5,10,21", 12),
    ("10,20", 15),
    ("5,,10", None),
    ("", None),
    ("-5,10", None),
])
def test_processing_average_correctness(input_data, expected):
    assert processing_average(input_data) == expected


@pytest.mark.parametrize("input_data, expected", [
    ("6000|500", 6580.0),
    ("1000|200", 980.0),
    ("abc|500", None),
    ("|", None),
    ("-100|0", None),
])
def test_payable_fee_correctness(input_data, expected):
    assert payable_fee(input_data) == expected
```